```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.13/dist-packages (from opencv-python) (2.1.3)
```

```
import cv2
import numpy as np
import time
from google.colab.patches import cv2_imshow
```

```
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode

def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
    async function takePhoto(quality) {
      const div = document.createElement('div');
      const video = document.createElement('video');

      video.style.display = 'block';
      const stream = await navigator.mediaDevices.getUserMedia({
        video: true
      });

      document.body.appendChild(div);
      div.appendChild(video);

      video.srcObject = stream;
      await video.play();

      await new Promise(resolve => setTimeout(resolve, 1000));

      const canvas = document.createElement('canvas');
      canvas.width = video.videoWidth;
      canvas.height = video.videoHeight;

      canvas.getContext('2d').drawImage(video, 0, 0);

      stream.getVideoTracks()[0].stop();
      div.remove();

      return canvas.toDataURL('image/jpeg', quality);
    }
    ''')

    display(js)

    data = eval_js(
        'takePhoto({})'.format(quality)
    )

    binary = b64decode(data.split(',')[1])

    with open(filename, 'wb') as f:
        f.write(binary)

    return filename
```

```
filename = take_photo()

print("Photo captured:", filename)

img = cv2.imread(filename)

cv2_imshow(img)
```

```
Photo captured: photo.jpg
```



```
first_photo = take_photo("first_frame.jpg")

first_frame = cv2.imread(first_photo)

print("First frame captured successfully!")

cv2_imshow(first_frame)
```

```
First frame captured successfully!
```



```
second_photo = take_photo("second_frame.jpg")

second_frame = cv2.imread(second_photo)

print("Second frame captured successfully!")

cv2_imshow(second_frame)
```

```
Second frame captured successfully!
```



```
gray1 = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(second_frame, cv2.COLOR_BGR2GRAY)

difference = cv2.absdiff(gray1, gray2)

_, threshold = cv2.threshold(
    difference,
    25,
    255,
    cv2.THRESH_BINARY
)

threshold = cv2.dilate(
    threshold,
    None,
    iterations=2
)

print("Motion difference:")
cv2_imshow(threshold)
```

```
Motion difference:
```



```
motion_pixels = cv2.countNonZero(threshold)

print("Motion pixels:", motion_pixels)

if motion_pixels > 5000:
```

```
    print("🚨 MOTION DETECTED!")
else:
    print("No significant motion detected.")
```

```
Motion pixels: 103035
🚨 MOTION DETECTED!
```

```
if motion_pixels > 5000:
    cv2.imwrite(
        "emergency_snapshot.jpg",
        second_frame
    )

    print("🚨 ALERT!")
    print("Emergency snapshot saved.")
```

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.13/dist-packages (from opencv-python) (2.1.3)
```

```
import cv2
import csv
from datetime import datetime
from google.colab.patches import cv2_imshow
```

```
qr_detector = cv2.QRCodeDetector()

print("QR Code Detector Ready!")
```

```
QR Code Detector Ready!
```

```
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode

def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
    async function takePhoto(quality) {
        const div = document.createElement('div');
        const video = document.createElement('video');

        video.style.display = 'block';

        const stream = await navigator.mediaDevices.getUserMedia({
            video: true
        });

        document.body.appendChild(div);
        div.appendChild(video);

        video.srcObject = stream;
        await video.play();

        await new Promise(resolve => setTimeout(resolve, 1000));

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;

        canvas.getContext('2d').drawImage(video, 0, 0);

        stream.getVideoTracks()[0].stop();
        div.remove();

        return canvas.toDataURL('image/jpeg', quality);
    }
    ''')

    display(js)

    data = eval_js(
        'takePhoto({})'.format(quality)
    )

    binary = b64decode(data.split(',')[1])

    with open(filename, 'wb') as f:
        f.write(binary)

    return filename

print("Camera function ready!")
```

```
Camera function ready!
```

```
filename = take_photo("qr_image.jpg")

img = cv2.imread(filename)

print("Image captured!")
cv2_imshow(img)
```

```
Image captured!
```



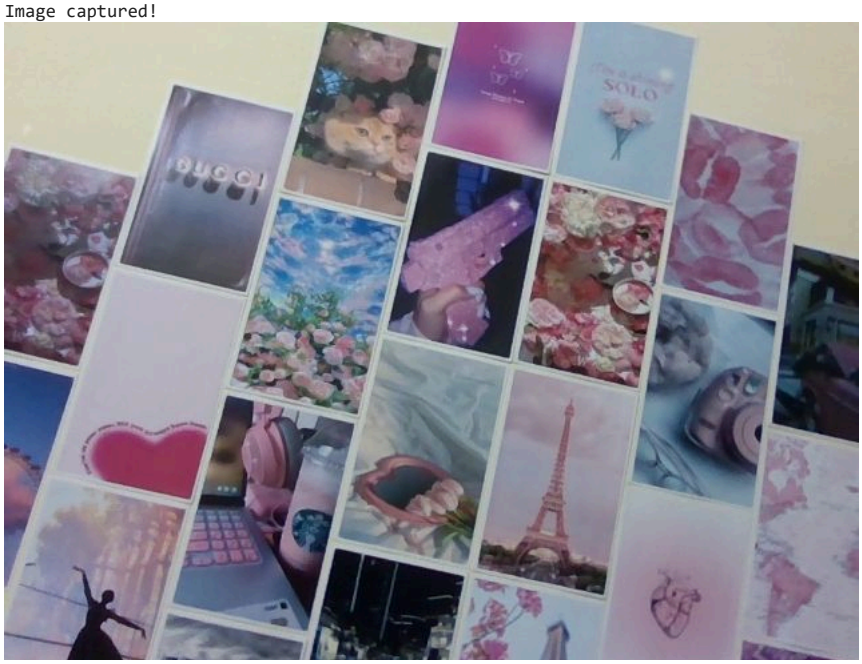
```
filename = take_photo("qr_image.jpg")

img = cv2.imread(filename)

print("Image captured!")

cv2_imshow(img)
```

```
Image captured!
```



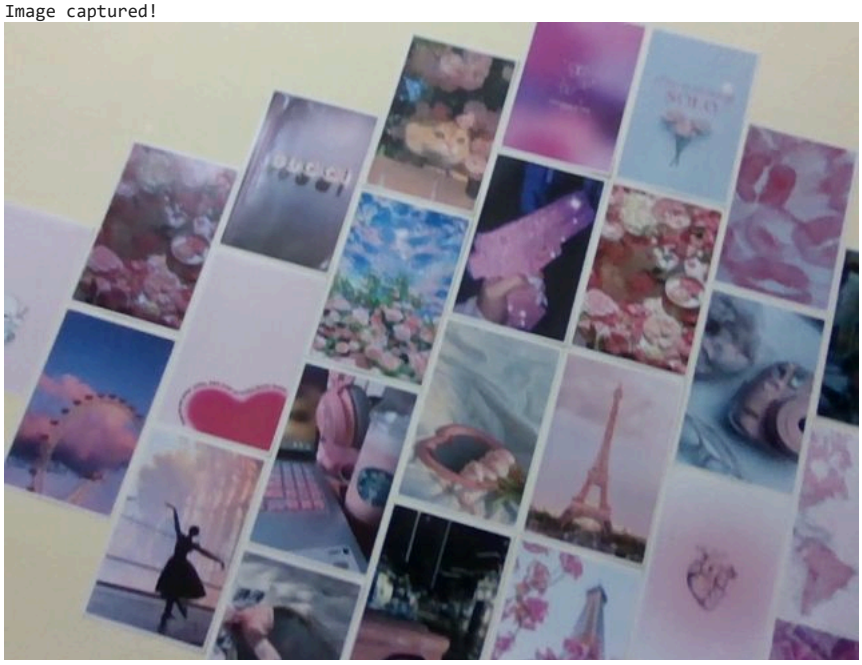
```
filename = take_photo("qr_image.jpg")

img = cv2.imread(filename)

print("Image captured!")

cv2_imshow(img)
```

```
Image captured!
```



```
data, points, _ = qr_detector.detectAndDecode(img)

if data:
    print("✅ QR CODE DETECTED")
    print("QR Data:", data)
else:
    print("❌ No QR code detected.")
```

```
❌ No QR code detected.
```

```
if data and points is not None:

    points = points.astype(int)

    for i in range(4):
        cv2.line(
            img,
            tuple(points[0][i]),
            tuple(points[0][(i + 1) % 4]),
            (0, 255, 0),
            3
        )

    cv2.putText(
        img,
        "QR DETECTED",
        (20, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        (0, 255, 0),
        2
    )

cv2_imshow(img)
```



```
from datetime import datetime
import csv

if data:

    timestamp = datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S"
    )

    with open(
        "inventory_log.csv",
        "a",
        newline="",
        encoding="utf-8"
    ) as file:

        writer = csv.writer(file)
```